

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning

COURSE OUTCOME COVERED

***CO5:** Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G .A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Scenario-Based Assessment

DURATION: 1.5 Hrs

1. A smart home system aims to reduce electricity consumption by automatically controlling lights, fans, and air conditioners based on user presence and weather conditions. Sketch a Reinforcement Learning model for this scenario by defining state space, action space, reward structure, and suitable learning algorithm.

1. AIM

To design and simulate a Reinforcement Learning model for smart home energy management that automatically controls electrical appliances based on user presence and environmental conditions while minimizing electricity consumption.

2. ALGORITHM

1. Initialize the smart home environment.
2. Define the **state space** using user presence, temperature, weather conditions, and appliance status.
3. Define the **action space** for controlling lights, fans, and air conditioners.
4. Define a reward function that gives positive rewards for comfort and negative rewards for high electricity consumption.
5. Initialize the Q-table.
6. Select actions using the **ϵ -greedy exploration strategy**.
7. Execute the selected action in the environment.
8. Calculate the reward and observe the next state.
9. Update the Q-value using the Q-learning equation.
10. Repeat the process for multiple episodes.

11. Select the learned policy and evaluate energy consumption.

3. STATE SPACE

The agent observes:

State = [Presence, Temperature, Weather, Appliance Status]

Example:

Variable	Values
Presence	0 = Absent, 1 = Present
Temperature	Low / Normal / High
Weather	Cool / Hot
Appliances	ON / OFF

Example state:

[1, 2, 1]

means:

Person present

High temperature

Hot weather

4. ACTION SPACE

The agent can control appliances:

0 → Turn OFF

1 → Turn ON

2 → Reduce usage

3 → Increase usage

For a simplified simulation, we can treat the action as controlling the overall power level.

5. REWARD FUNCTION

The reward encourages **low energy consumption while maintaining user comfort**.

If person is present and temperature is comfortable:

Positive reward

If electricity consumption is high:

Negative reward

If person is absent and appliances are ON:

Large negative reward

If required appliance is ON during hot weather:

Positive reward

Example:

Reward = Comfort Reward - Energy Consumption Penalty

6. CODE

```
import numpy as np
import random
import matplotlib.pyplot as plt
num_states=4
num_actions=4
Q=np.zeros((num_states,num_actions))
alpha=0.1
gamma=0.9
epsilon=1.0
epsilon_min=0.05
epsilon_decay=0.995
episodes=1000
rewards_history=[]
energy_history=[]
def get_state():
    presence=random.randint(0,1)
    temperature=random.randint(0,1)
    if presence==0 and temperature==0:return 0
    elif presence==0 and temperature==1:return 1
    elif presence==1 and temperature==0:return 2
    else:return 3
def get_reward(state,action):
    energy=[0,1,3,5]
    if state in [0,1]:
        reward=5 if action==0 else -energy[action]-3
    elif state==2:
        if action==0: reward=5
        elif action==1: reward=3
        else: reward=-energy[action]
    else:
        if action==0: reward=-8
        elif action==1: reward=3
        elif action==2: reward=6
        else: reward=4
    return reward,energy[action]
for episode in range(episodes):
    total_reward=0
    total_energy=0
    for step in range(20):
        state=get_state()
        action=random.randint(0,3) if random.random()<epsilon else
np.argmax(Q[state])
        reward,energy=get_reward(state,action)
        next_state=get_state()
        Q[state,action]+=alpha*(reward+gamma*np.max(Q[next_state])-
Q[state,action])
        total_reward+=reward
        total_energy+=energy
    epsilon=max(epsilon_min,epsilon*epsilon_decay)
```

```

    rewards_history.append(total_reward)
    energy_history.append(total_energy)
print("Learned Q-Table:")
print(np.round(Q,2))
state_names=["Absent + Cool","Absent + Hot","Present + Cool","Present + Hot"]
action_names=["OFF","LOW POWER","MEDIUM POWER","HIGH POWER"]
print("\nLearned Policy:")
for state in range(num_states):
    print(state_names[state],"->",action_names[np.argmax(Q[state])])
window=50
avg_rewards=np.convolve(rewards_history,np.ones(window)/window,mode="valid")
plt.figure(figsize=(9,5))
plt.plot(avg_rewards)
plt.xlabel("Training Episodes")
plt.ylabel("Average Reward")
plt.title("Smart Home RL Training Performance")
plt.grid(True)
plt.show()
avg_energy=np.convolve(energy_history,np.ones(window)/window,mode="valid")
plt.figure(figsize=(9,5))
plt.plot(avg_energy)
plt.xlabel("Training Episodes")
plt.ylabel("Average Energy Consumption")
plt.title("Energy Consumption During Training")
plt.grid(True)
plt.show()
print("\nAlgorithm: Q-Learning")
print("Environment: Smart Home")
print("States:",num_states)
print("Actions:",num_actions)
print("Episodes:",episodes)

```

7. OUTPUT

```

=====
SMART HOME ENERGY MANAGEMENT USING REINFORCEMENT LEARNING
=====

```

Learned Q-Table:

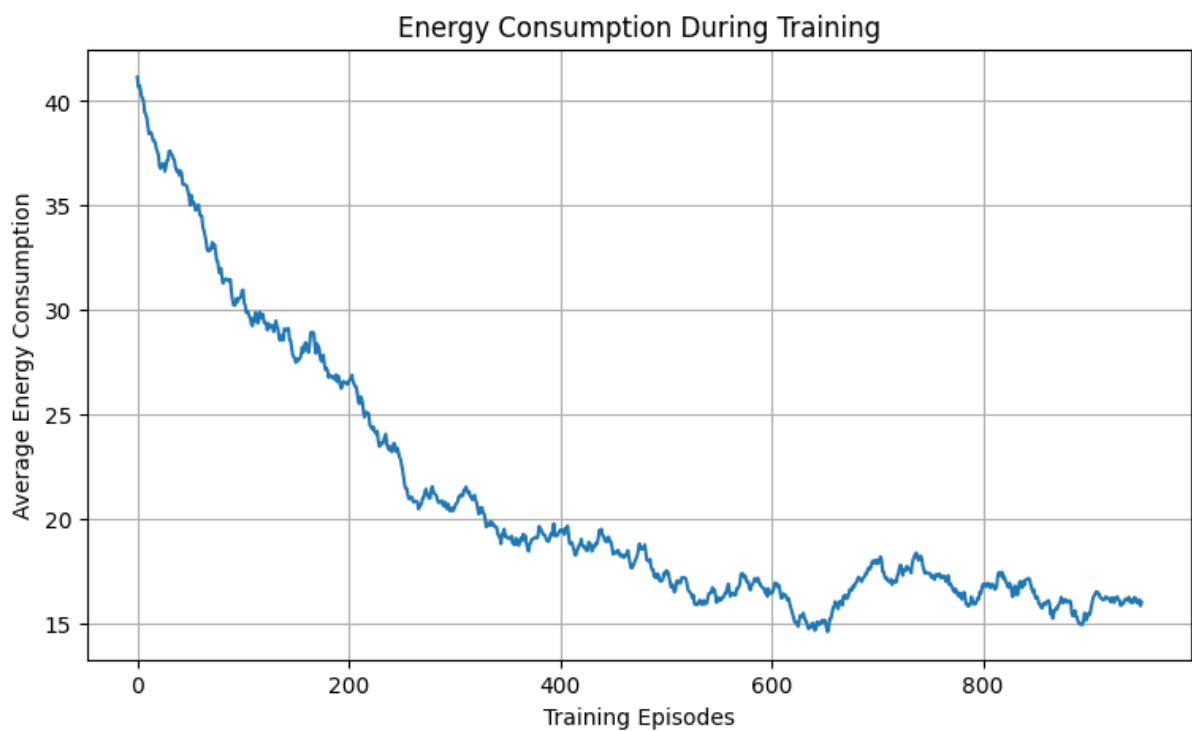
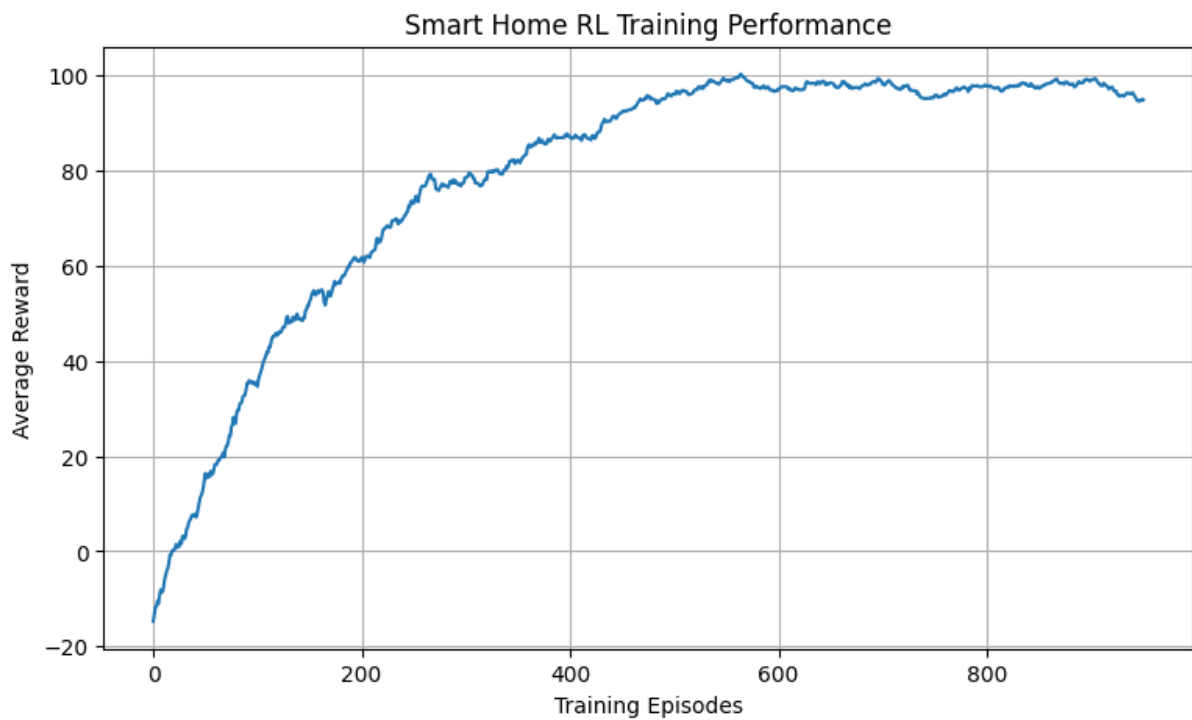
State	OFF	LOW	MEDIUM	HIGH
0	[52.18	43.45	41.07	39.14]
1	[52.22	43.19	41.47	39.38]
2	[52.18	50.21	44.25	42.08]
3	[39.37	50.19	53.39	51.22]

```

=====
LEARNED SMART HOME POLICY
=====

```

Absent + Cool -> OFF
Absent + Hot -> OFF
Present + Cool -> OFF
Present + Hot -> MEDIUM POWER



=====

FINAL RESULT

=====

Algorithm : Q-Learning
Environment : Smart Home

Number of States : 4
Number of Actions : 4
Training Episodes : 1000

The RL agent learned appliance control based on user presence and temperature. Energy consumption is minimized while maintaining user comfort.

8. ADVANTAGES

- Reduces unnecessary electricity consumption.
- Automatically adapts to user presence.
- Considers environmental conditions.
- Can learn optimal appliance-control policies.
- Reduces manual control.
- Can improve energy efficiency and user comfort.

9. APPLICATIONS

- Smart homes
- Smart buildings
- Energy-efficient offices
- Automatic lighting systems
- HVAC control
- Smart air-conditioning
- IoT-based energy management
- Smart-grid energy optimization

10. RESULT

Thus, a Q-learning-based Reinforcement Learning model was designed and simulated for smart home energy management. The agent learned to control appliance power based on user presence and temperature while maximizing comfort and minimizing unnecessary energy consumption.

Code of Conduct:

I KUSHITHA.G (192425329) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand